

Lernbericht

DeskBuddy System

Gemeinsamer Lernbericht der Module PROG3 und ADP

HF Informatik · gibb Bern · 2026

Venurshan Manivannan

Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Einleitung	3
2. Ausgangslage und Vorkenntnisse	3
3. Persönliche Lernziele	3
4. PROG3: Technischer Lernprozess	3
4.1 Backend	3
4.2 Frontend	4
4.3 ESP32	4
4.4 Google Calendar	4
4.5 Testing.....	4
5. ADP: Agiler Entwicklungsprozess.....	4
5.1 Methode	4
5.2 Sprintplanung	5
5.3 Board und Akzeptanzkriterien	5
5.4 Git und Commits	5
5.5 Reviews und Fortschritt.....	5
6. Schwierigkeiten und Lösungen	5
7. Reflexion und alternative Vorgehensweisen	6
8. Einsatz von KI-Werkzeugen	6
9. Transfer in zukünftige Projekte und Berufspraxis	6
10. Fazit und Bewertung meiner eigenen Leistung	7

Hinweis: In Word das Inhaltsverzeichnis anklicken und mit F9 aktualisieren, um Seitenzahlen einzutragen.

1. Einleitung

Im Projekt DeskBuddy System habe ich eine verteilte Anwendung umgesetzt, die Kalenderdaten aus Google Calendar verarbeitet und über ein Backend, ein React-Dashboard und ein ESP32-Gerät nutzbar macht. Ziel war nicht, einen vollständigen Kalender nachzubauen, sondern die wichtigsten Informationen kompakt sichtbar zu machen: welcher Termin gerade läuft und was als Nächstes kommt.

Dieser Lernbericht betrachtet das Projekt aus zwei Perspektiven. Im technischen Teil steht PROG3 im Fokus, also Architektur, Backend, Frontend, Datenbank, Testing und die Anbindung der Hardware. Im methodischen Teil steht ADP im Fokus, also Planung, Sprintstruktur, Kanban-Board, Akzeptanzkriterien, Commits und die Reflexion meines agilen Vorgehens. Beide Teile sind klar gekennzeichnet, lassen sich aber als ein zusammenhängender Bericht lesen.

2. Ausgangslage und Vorkenntnisse

Vor diesem Projekt hatte ich bereits Grundlagen in der Webentwicklung mit JavaScript, in der Arbeit mit REST-APIs und im Umgang mit relationalen Datenbanken. Mit C# und ASP.NET Core hatte ich erste Erfahrungen, aber noch keine vollständige Anwendung von der Datenbank bis zum Frontend gebaut. Mit Entity Framework Core und JWT-Authentifizierung hatte ich nur theoretisch zu tun gehabt.

Neu waren für mich vor allem drei Dinge: die Integration einer externen API über OAuth2, die Anbindung eigener Hardware (ESP32) an ein selbst geschriebenes Backend und das durchgängige Testen über mehrere Ebenen hinweg. Diese drei Punkte waren entsprechend die grössten Lernfelder.

3. Persönliche Lernziele

Für das Projekt habe ich mir folgende Lernziele gesetzt:

- Eine vollständige Web-API mit ASP.NET Core und EF Core selbstständig aufbauen, inklusive Datenbank und Persistenz.
- CRUD und eine saubere DTO-Schicht so umsetzen, dass interne Daten nicht ungewollt nach aussen gelangen.
- Zwei Authentifizierungsverfahren (JWT und API-Key) verstehen und sinnvoll einsetzen.
- Eine externe API (Google Calendar) über OAuth2 anbinden.
- Ein Frontend und ein physisches Gerät als Clients an dieselbe API anschliessen.
- Ein nachvollziehbares Testkonzept aufbauen und automatisierte Tests schreiben.
- Mein Vorgehen agil strukturieren und den Fortschritt sichtbar dokumentieren (ADP).

4. PROG3: Technischer Lernprozess

MODUL PROG3 – TECHNISCHE UMSETZUNG

In diesem Kapitel reflektiere ich die technische Umsetzung. Ich gehe auf die einzelnen Komponenten ein und beschreibe, was ich dabei gelernt habe und was gut oder weniger gut lief.

4.1 Backend

Das Backend habe ich mit ASP.NET Core 8 und Entity Framework Core 8 auf einer SQLite-Datenbank umgesetzt. SQLite habe ich gewählt, weil das System lokal für ein einzelnes Gerät läuft und keine grosse Datenmenge anfällt. Hier habe ich am meisten über den Aufbau einer API gelernt: wie Controller, Services und das Datenmodell zusammenspielen und wie man die Logik sinnvoll trennt.

CRUD habe ich für die zwei Objekte Device und CalendarEvent umgesetzt. Zwischen den internen Modellen und der API liegt eine DTO-Schicht. Anfangs war mir der Nutzen von DTOs nicht ganz klar, aber spätestens beim API-Key des Geräts wurde es deutlich: Der Key ist intern gespeichert, soll aber nie in einer Antwort auftauchen. Über das DTO konnte ich genau steuern, was nach aussen geht. Das war für mich ein wichtiger Aha-Moment.

Bei der Authentifizierung habe ich zwei Wege umgesetzt, weil ein Mensch und ein Gerät sich unterschiedlich anmelden. Das Dashboard nutzt ein JWT, das Gerät einen festen API-Key. Diese Unterscheidung zu verstehen und beides parallel zum Laufen zu bringen, war lehrreich, besonders das Zusammenspiel von Login, Token und geschützten Endpunkten.

4.2 Frontend

Das Dashboard habe ich mit React, Vite und Tailwind CSS gebaut. Da ich aus der Webentwicklung schon Grundlagen mitbrachte, fiel mir dieser Teil etwas leichter. Trotzdem habe ich dazugelernt, vor allem beim Umgang mit dem JWT im Client, beim automatischen Neu-Anmelden nach Ablauf des Tokens und beim regelmässigen Nachladen der Daten. Die einzelnen Seiten zeigen den Gerätestatus, die Termine und die aktuelle Now/Next-Ansicht.

4.3 ESP32

Die Anbindung des ESP32-S3 mit AMOLED-Display war für mich der ungewohnteste Teil, weil ich vorher kaum mit eigener Hardware gearbeitet hatte. Das Gerät sendet regelmässig einen Heartbeat und fragt die Now/Next-Daten ab, um sie auf dem Display anzuzeigen. Rückblickend habe ich das Gerät zu spät ins Projekt geholt (siehe Kapitel 6). Als es dann lief und der erste echte Termin auf dem Display erschien, war das aber der motivierendste Moment im ganzen Projekt.

4.4 Google Calendar

Die Termine kommen über die Google Calendar API ins System, angebunden über OAuth2 mit reinem Lesezugriff. Der OAuth2-Flow war anfangs schwierig zu verstehen, weil ein einmaliger Login im Browser nötig ist und das Token danach lokal weiterverwendet wird. Hier musste ich mich einarbeiten und einiges nachlesen, bevor es zuverlässig funktionierte.

4.5 Testing

Ich habe auf mehreren Ebenen getestet: Unit-Tests für die Now/Next-Logik, Integrationstests für die ganze API gegen eine In-Memory-Datenbank, Postman-Tests für die echten Endpunkte sowie manuelle Tests für Frontend und Gerät. Dass die automatisierten Tests ohne laufenden Server und ohne echte Google-Verbindung durchlaufen, fand ich besonders nützlich. Gelernt habe ich vor allem, dass sich gut getrennte, reine Logik (wie die Now/Next-Berechnung) deutlich einfacher testen lässt. Das hat meine Art, Code zu strukturieren, beeinflusst.

5. ADP: Agiler Entwicklungsprozess

MODUL ADP – AGILES VORGEHEN

5.1 Methode

Da ich alleine gearbeitet habe, habe ich einen vereinfachten Scrum-/Kanban-Ansatz gewählt. Ein vollständiger Scrum-Prozess mit Rollen wie Product Owner oder Scrum Master wäre für eine Einzelperson übertrieben gewesen. Stattdessen habe ich die nützlichen Elemente übernommen: feste Sprints mit Zielen, ein Board zur Visualisierung und regelmässige Rückblicke.

5.2 Sprintplanung

Die Arbeit habe ich in sieben Sprints von 0 bis 6 aufgeteilt. Sprint 0 diente dem Setup und der Einrichtung des Boards, danach folgten Backend-Grundgerüst, CRUD und DTOs, Authentifizierung, Google-Calendar-Integration, Frontend und zuletzt Testing und Dokumentation. Jeder Sprint hatte einen klaren Schwerpunkt, der auf dem vorherigen aufbaute. Diese Struktur hat mir geholfen, den Überblick zu behalten und nicht alles gleichzeitig anzugehen.

5.3 Board und Akzeptanzkriterien

Verwaltet habe ich die Aufgaben auf einem Notion-Kanban-Board. Insgesamt waren es 82 Tasks, jeweils mit Status, Sprint, Kategorie und Priorität. Zu vielen Tasks habe ich Akzeptanzkriterien notiert, zum Beispiel „Login liefert bei korrekten Daten ein Token“ oder „Gerät sendet alle 30 Sekunden einen Heartbeat“. Diese Kriterien haben mir geholfen, eine Aufgabe erst dann als erledigt zu markieren, wenn sie wirklich fertig war, und nicht nur „gefühlte“ fertig. Die Priorisierung habe ich genutzt, um wichtige Grundlagen (zuerst Backend und Datenmodell) vor optionalen Verbesserungen zu erledigen.

5.4 Git und Commits

Die Commits habe ich mit den Aufgaben im Board verknüpft, sodass sich zu jeder Aufgabe die passenden Code-Änderungen nachvollziehen lassen. Da ich alleine gearbeitet habe, habe ich bewusst auf einen komplexen Branch-Workflow verzichtet und hauptsächlich auf dem main-Branch gearbeitet, dafür aber regelmässig committet. Mir ist bewusst, dass das in einem Team anders aussehen müsste; bei mehreren Personen würde ich konsequent mit Branches und Pull Requests arbeiten. Für ein Einzelprojekt war der einfache Ansatz aber angemessen und hat die Nachvollziehbarkeit nicht beeinträchtigt.

5.5 Reviews und Fortschritt

Nach jedem Sprint habe ich einen kurzen Rückblick gemacht und geprüft, was erreicht wurde und was als Nächstes ansteht. Der Fortschritt war über das Board jederzeit sichtbar, weil sich erledigte Tasks in die Spalte Done verschoben haben. Am Ende waren alle 82 Tasks abgeschlossen. Diese Sichtbarkeit hat mich motiviert und mir geholfen, den Zeitplan einzuschätzen.

6. Schwierigkeiten und Lösungen

Nicht alles lief reibungslos. Die folgenden Punkte waren die grössten Herausforderungen:

Google-Sync und EF Core: Beim Aktualisieren der Termine (Upsert) gab es Tracking-Probleme in EF Core. Gelöst habe ich das mit einem Full-Replace-Ansatz: Beim Sync werden alle alten Termine gelöscht und neu eingefügt. Das ist einfacher und vermeidet doppelte oder veraltete Einträge.

OAuth2-Flow: Der Einstieg in OAuth2 war anfangs schwierig, vor allem der einmalige Browser-Login und die Weiterverwendung des Tokens. Mit Nachlesen und Ausprobieren habe ich es zum Laufen gebracht.

Hardware zu spät integriert: Das ESP32-Gerät habe ich relativ spät angebunden. Dadurch entstand gegen Ende Zeitdruck, weil sich kleinere Hardware-Probleme schlecht einplanen liessen.

Secrets und Konfiguration: Zugangsdaten und Keys könnten zentraler verwaltet werden. Aktuell liegen die Admin-Daten in der Konfigurationsdatei (nur für die Entwicklung), und die Geräte-ID ist im Frontend teilweise fest hinterlegt. Das funktioniert, ist aber nicht die sauberste Lösung.

7. Reflexion und alternative Vorgehensweisen

Rückblickend würde ich einige Dinge anders machen. Diese Alternativen sehe ich als konkrete Verbesserungen für ein nächstes Projekt:

- Die Hardware früher mit einem kleinen Proof of Concept testen, statt sie erst gegen Ende einzubinden.
- Secrets konsequenter über Umgebungsvariablen oder einen Secret-Storage verwalten, statt sie in Konfigurationsdateien zu halten.
- Automatisierte Tests früher einbauen, statt einen grossen Teil davon erst im letzten Sprint zu schreiben.
- Die Gerätedaten im Frontend dynamisch laden, statt eine feste Geräte-ID zu verwenden.
- In einem Team einen konsequenten Branch-Workflow (z. B. GitFlow) mit Pull Requests einsetzen.

Insgesamt bin ich mit dem Vorgehen zufrieden. Die agile Struktur hat funktioniert, und die Probleme, die auftraten, konnte ich nachvollziehbar lösen. Wichtiger als ein perfektes Ergebnis war für mich, dass ich die Entscheidungen verstanden habe und begründen kann.

8. Einsatz von KI-Werkzeugen

Während des Projekts habe ich KI-Werkzeuge als Unterstützung genutzt, vor allem für Code-Vorschläge, das Strukturieren von Ideen, das Debugging und das Formulieren von Texten. Wichtig ist mir, das ehrlich zu benennen.

Gleichzeitig habe ich die Entscheidungen selbst getroffen und nachvollzogen. Die KI hat mir geholfen, schneller voranzukommen, aber das Verständnis, die Anpassungen am Code, die Tests, die Präsentation und die Einordnung der Ergebnisse stammen von mir. Vorschläge habe ich nicht blind übernommen, sondern geprüft, angepasst und in vielen Fällen verworfen, wenn sie nicht passten. Ich kann jeden Teil des Systems erklären und begründen, warum ich ihn so gebaut habe.

9. Transfer in zukünftige Projekte und Berufspraxis

Aus dem Projekt nehme ich mehrere Dinge mit, die ich in zukünftigen Projekten und im Beruf anwenden kann. Technisch habe ich gelernt, eine API von der Datenbank bis zum Client durchgängig aufzubauen und dabei auf eine saubere Trennung von Logik, Datenmodell und nach aussen sichtbarer Schnittstelle zu achten. Die Erfahrung, dass gut testbarer Code auch besser strukturierter Code ist, werde ich beibehalten.

Methodisch nehme ich mit, dass auch ein vereinfachtes agiles Vorgehen viel bringt: feste Sprints, ein sichtbares Board und Akzeptanzkriterien helfen, fokussiert und ehrlich mit dem eigenen Fortschritt umzugehen. In einem Team würde ich diese Grundidee um einen sauberen Branch- und Review-Prozess ergänzen. Auch der bewusste und ehrliche Umgang mit KI-Werkzeugen ist etwas, das ich in die Berufspraxis übernehmen möchte.

10. Fazit und Bewertung meiner eigenen Leistung

Ich habe meine Lernziele weitgehend erreicht. Das System läuft als Ganzes: Das Backend stellt die Daten bereit, das Dashboard und das Gerät zeigen sie an, und die Termine kommen automatisch aus Google Calendar. CRUD, DTOs, beide Authentifizierungsverfahren und ein mehrstufiges Testkonzept sind umgesetzt. Im agilen Teil habe ich das Projekt durchgehend strukturiert geplant, dokumentiert und reflektiert.

Gleichzeitig sehe ich klar, wo es Grenzen gibt: Das System läuft nur lokal, einige Konfigurationswerte sind fest hinterlegt, und einen Teil der Tests habe ich erst spät geschrieben. Diese Punkte verschweige ich nicht, sondern sehe sie als konkrete Ansätze für das nächste Mal.

Meine eigene Leistung bewerte ich als gut. Der grösste Wert lag für mich nicht darin, dass alles perfekt funktioniert, sondern dass ich ein vollständiges System über mehrere Technologien hinweg selbst aufgebaut, die auftretenden Probleme verstanden und gelöst und das Vorgehen ehrlich reflektiert habe. Genau das nehme ich als wichtigsten Lernerfolg mit.